



**Politecnico
di Torino**

Statlog (Vehicle Silhouettes) Dataset: un'analisi mediante tecniche di Supervised e Unsupervised learning

Corso di Laurea in Matematica per l'Ingegneria
Matematica per l'Intelligenza Artificiale

Matteo Ferrero - s307402

Anno accademico 2025-2026

Indice

1	Analisi del dataset	2
2	Riduzione di dimensionalità	6
2.1	PCA	6
2.2	MDA	11
3	Classificazione	15
3.1	KNN	16
3.2	LDA	19
3.3	SVM	22
3.3.1	Kernel SVM	23
3.3.2	SVM multiclasse	23
3.4	MLP	26
4	Conclusioni	31
4.1	Bibliografia	31

Capitolo 1

Analisi del dataset

Il dataset in esame è composto da $N = 846$ veicoli di 4 modelli differenti: un bus double decker, un van della Chevrolet, la Saab 9000 e la Opel Manta 400. Ad ogni veicolo sono associate 18 *features* riguardanti la silhouette dei veicoli estratte direttamente dalle immagini dei veicoli fatte tutte con la stessa angolazione per poter essere confrontate correttamente. Inoltre, per rendere le features indipendenti da altri fattori esterni non geometrici, i veicoli sono stati dipinti di nero prima di ottenerne le immagini.

Il processo di estrazione delle features dalle silhouette dei veicoli si è avvalso dell'estensione BINATTS, integrata nel sistema di elaborazione delle immagini HIPS (*Hierarchical Image Processing System*), in modo tale che risultino indipendenti dalla scala dell'immagine. Le features si possono inoltre suddividere in due tipologie principali:

- **Misure statistiche classiche:** tra cui varianza, skewness (asimmetria) e curtosi (appiattimento) determinate lungo gli assi principali e secondari della figura
- **Misure euristiche e morfologiche:** volte a catturare la geometria strutturale del veicolo come la compattezza, la circolarità, la rettangolarità e la presenza di cavità

E' inoltre importante da sottolineare che nel dataset originale erano presenti 100 dati in più usati come test set, che però non sono stati pubblicati assieme al resto del dataset

Il dataset può essere quindi scomposto in una matrice $X \in \mathbb{R}^{N \times 18}$ che ha per righe i veicoli e sulle colonne il valore di ogni feature associata al veicolo. Nello specifico le features hanno il seguente significato:

Feature	Descrizione
Compactness	Compattezza media della sagoma
Circularity	Misura di quanto la sagoma si avvicina a un cerchio.
Distance Circularity	Circolarità calcolata come rapporto tra l'area del veicolo e la distanza media dei punti dal bordo
Radius Ratio	Rapporto tra il raggio massimo e minimo della sagoma
Pr.axis Aspect Ratio	Rapporto tra l'asse maggiore e l'asse minore dell'ellisse approssimante.
Max. Length Aspect Ratio	Rapporto tra la lunghezza massima e la larghezza della sagoma.
Scatter Ratio	Rapporto tra la distribuzione dei punti lungo l'asse minore e lungo l'asse maggiore
Elongatedness	Grado di allungamento della figura
Pr.axis Rectangular	Grado con cui si può approssimare la sagoma con il rettangolo costruito sugli assi (retticolarità)
Max.length Rectangularity	Misura di quanto la sagoma riempie il suo rettangolo contenitore
Major Variance	Varianza dei punti lungo l'asse maggiore
Minor Variance	Varianza dei punti lungo l'asse minore
Gyration Radius	Raggio di girazione (distanza media dei punti dal centro di massa).
Skewness Major	Asimmetria della distribuzione dei punti lungo l'asse maggiore
Skewness Minor	Asimmetria della distribuzione dei punti lungo l'asse minore
Kurtosis Major	Appiattimento (curtosi) della distribuzione lungo l'asse maggiore
Kurtosis Minor	Appiattimento (curtosi) della distribuzione lungo l'asse minore
Hollows Ratio	Rapporto tra l'area dei vuoti interni e l'area del poligono delimitante la sagoma

Tabella 1.1: Descrizione delle feature del dataset Statlog (Vehicle Silhouettes)

Le classi (tipologie di veicolo) verranno da qui in avanti chiamate *Opel*, *Saab*, *Bus* e *Van* e possono essere inserite in un vettore colonna $y \in \mathbb{N}^N$. Più nello specifico le classi hanno le seguenti caratteristiche in relazione alle feature:

Classe	Significato / Descrizione
Bus	Rappresenta i grandi mezzi di trasporto pubblico (autobus). Hanno solitamente valori alti di rettilinearità e dimensioni ampie.
Van	Rappresenta i furgoni o veicoli commerciali leggeri. Caratterizzati da una forma squadrata ma più compatta rispetto ai bus.
Saab	Si riferisce al modello di auto <i>Saab 9000</i> . Viene usata come esempio di berlina, quindi di auto compatta.
Opel	Si riferisce al modello di auto <i>Opel Manta</i> . Viene usata come esempio di auto sportiva/coupe, con profilo più basso e allungato.

Tabella 1.2: Descrizione delle classi target del dataset Vehicle

Innanzitutto scelgo di manipolare i miei dati utilizzando *Pandas*, in particolare inserendoli in un *DataFrame* e usando i metodi forniti da *pandas*

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 846 entries, 0 to 845
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   compactness                          846 non-null    int64
1   circularity                          846 non-null    int64
2   distance_circularity                 846 non-null    int64
3   radius_ratio                        846 non-null    int64
4   pr_axis_aspect_ratio                 846 non-null    int64
5   max_length_aspect_ratio              846 non-null    int64
6   scatter_ratio                       846 non-null    int64
7   elongatedness                       846 non-null    int64
8   pr_axis_rectangularity               846 non-null    int64
9   max_length_rectangularity            846 non-null    int64
10  scaled_variance_major                 846 non-null    int64
11  scaled_variance_minor                 846 non-null    int64
12  scaled_radius_of_gyration             846 non-null    int64
13  skewness_major                       846 non-null    int64
14  skewness_minor                       846 non-null    int64
15  kurtosis_minor                       846 non-null    int64
16  kurtosis_major                       846 non-null    int64
17  hollows_ratio                        846 non-null    int64
18  vehicle_class                        846 non-null    object
dtypes: int64(18), object(1)
memory usage: 125.7+ KB
```

Si può notare innanzitutto che il dataset è completo, ovvero per ogni colonna sono presenti 846 valori numerici leggibili. Analizzando poi i tipi delle features, si può notare che le classi sono espresse tramite variabile non intera: è utile invece per il nostro obiettivo codificare le classi con interi da 0 a 3 grazie al *LabelEncoder* di *ScikitLearn*

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 846 entries, 0 to 845
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   compactness                          846 non-null    int64
1   circularity                         846 non-null    int64
2   distance_circularity                846 non-null    int64
3   radius_ratio                       846 non-null    int64
4   pr_axis_aspect_ratio               846 non-null    int64
5   max_length_aspect_ratio            846 non-null    int64
6   scatter_ratio                      846 non-null    int64
7   elongatedness                      846 non-null    int64
8   pr_axis_rectangularity             846 non-null    int64
9   max_length_rectangularity          846 non-null    int64
10  scaled_variance_major              846 non-null    int64
11  scaled_variance_minor              846 non-null    int64
12  scaled_radius_of_gyration          846 non-null    int64
13  skewness_major                    846 non-null    int64
14  skewness_minor                    846 non-null    int64
15  kurtosis_minor                    846 non-null    int64
16  kurtosis_major                    846 non-null    int64
17  hollows_ratio                     846 non-null    int64
18  vehicle_class                      846 non-null    int64
dtypes: int64(19)
memory usage: 125.7 KB

```

Passiamo poi alla descrizione statistica delle features, ottenendo facilmente delle informazioni grazie al metodo `.describe()` di *Pandas* (per ragioni di spazio non ho potuto inserire un'immagine di output del metodo, ma è riportato nel notebook contenente i codici).

Si può osservare che, ad esempio, il valor medio della *compactness* è significativamente diverso da quello della *scaled_variance_minor* e in generale anche osservando minimi e massimi si nota che le features hanno ordini di grandezza differenti. Inoltre la feature *scaled_variance_minor* ha anche una varianza molto maggiore rispetto alle altre features: pertanto per non avere problemi nella PCA come perdere informazioni in termini di varianza sulle feature che ne hanno meno, sarà necessario riscalarle utilizzando lo *StandardScaler* di *ScikitLearn*.

In ultimo osserviamo la molteplicità di ogni classe nel dataset per capirne il bilanciamento

	Classe	Numerosità	Frequenza
0	bus	218	25.77
1	saab	217	25.65
2	opel	212	25.06
3	van	199	23.52

La frequenza di ogni classe si aggira attorno al 25%, questo significa che avendo 4 classi il dataset è bilanciato e non sarà necessario ricorrere a tecniche di ricampionamento. Per quanto riguarda la suddivisione del dataset, è importante tenere in conto che la numerosità dei dati è relativamente bassa, quindi la scelta sarà di dividerlo in: *training set* (70%) e *test set* (30%)

Capitolo 2

Riduzione di dimensionalità

Quando la dimensione dello spazio delle features è molto grande, siamo interessati a rappresentare i nostri dati in una dimensione ridotta per evitare problemi legati all'alta dimensionalità, anche se questo comporterà una perdita di informazione. Nel nostro caso specifico abbiamo i veicoli rappresentati tramite 18 features, quindi sono punti di $\mathbb{R}^{18}$, e vogliamo rappresentarli in uno spazio $\mathbb{R}^m$ con $m < 18$. La scelta delle tecniche in questo caso ricadrà su *Principal Component Analysis (PCA)* e *Multiple Discriminant Analysis (MDA)*

2.1 PCA

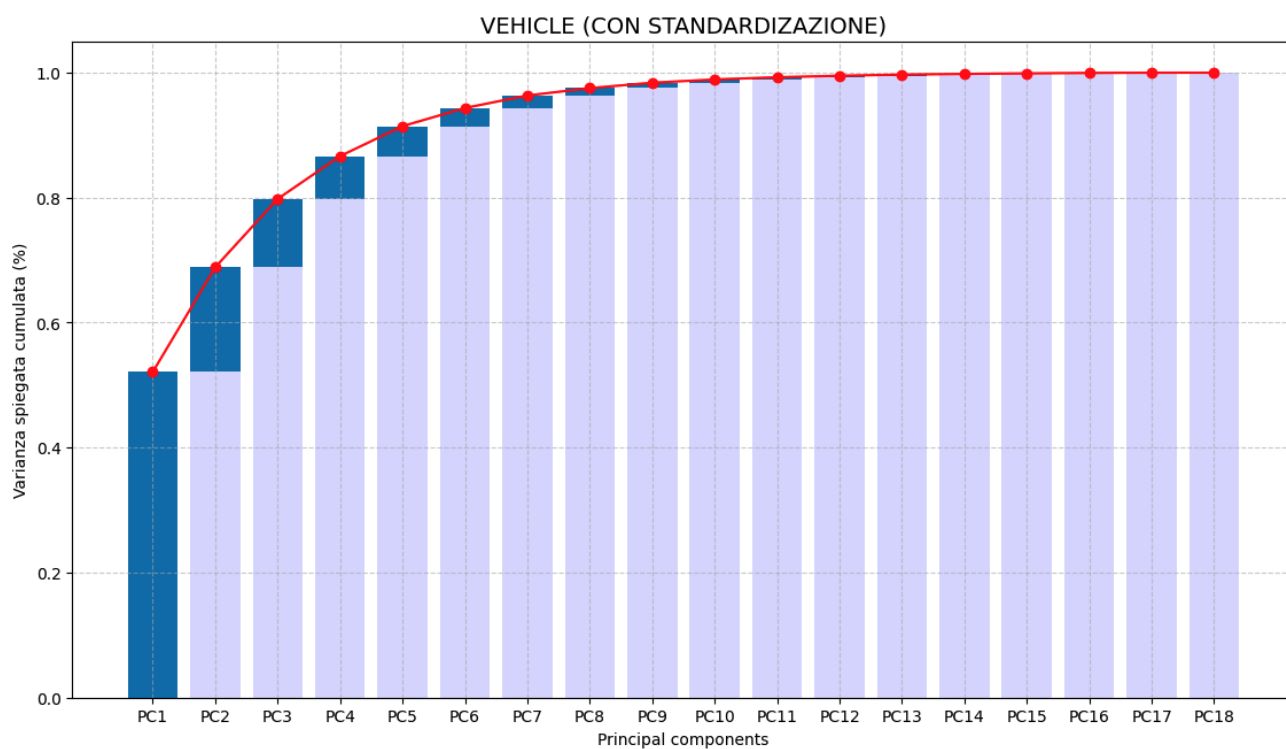
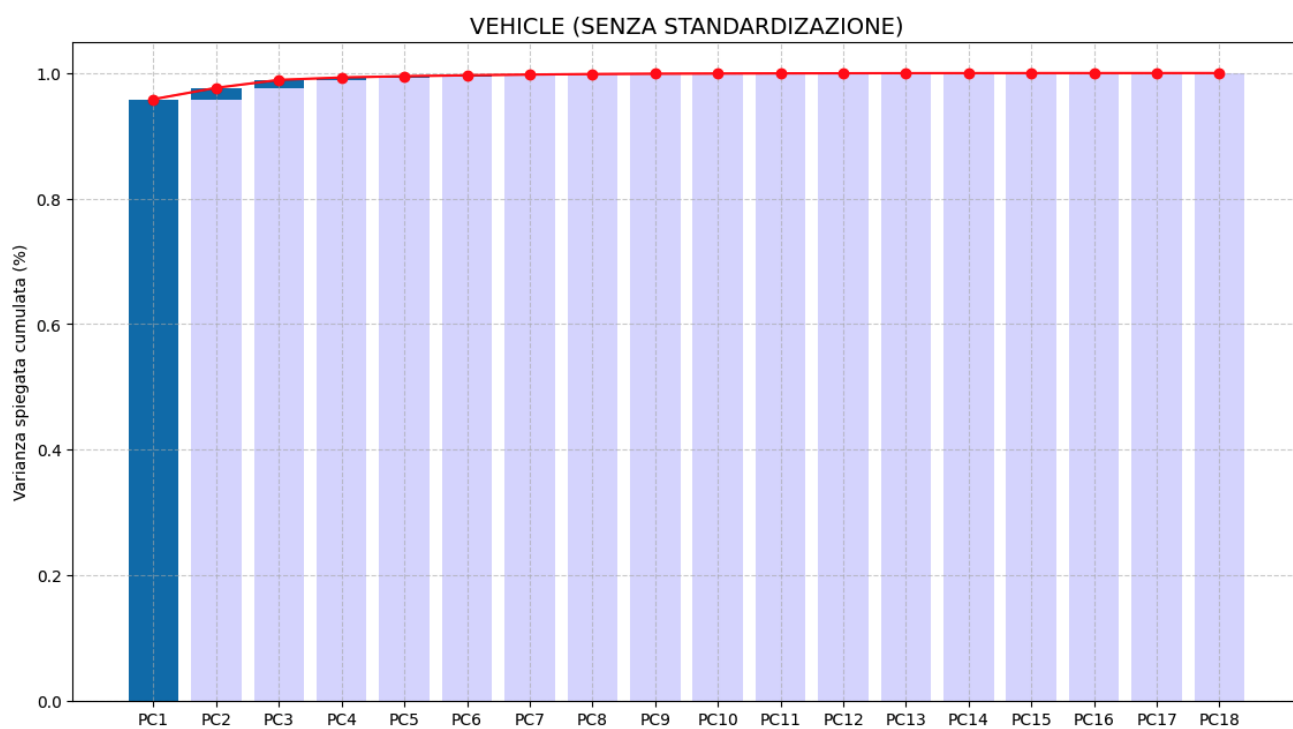
La PCA è uno strumento di riduzione dimensionale che, considerando i dati originali centrati nell'origine appartenenti a $\mathbb{R}^n$, cerca un sottospazio di dimensione $m < n$ su cui proiettare i dati grazie ad un'applicazione lineare $P \in \mathbb{R}^{n \times m}$ con l'obiettivo di massimizzare la varianza ottenuta rispetto all'originale dei dati. L'idea in particolare è che proiettando i dati $x_i \in \mathbb{R}^n$ si otterrà la sua rappresentazione nel sottospazio di dimensione m tramite $P^T x_i$, ma poi si può anche tornare indietro utilizzando l'applicazione lineare una seconda volta, ottenendo nuovamente un dato in $\mathbb{R}^n$: $\tilde{x}_i = P P^T x_i$, che tuttavia sarà inevitabilmente soggetto ad un errore e quindi in generale $x_i \neq \tilde{x}_i$: l'obiettivo del metodo a questo punto sarà proprio quello di minimizzare questo errore espresso tramite la funzione *loss* (scegliamo per comodità l'errore quadratico, ma non deve per forza essere quello)

$$L_P = \sum_{i=1}^n \|x_i - \tilde{x}_i\|_2^2$$

Le colonne di P sono m vettori ortogonali e li chiameremo proprio *componenti principali*. Questi vettori non sono altro che gli autovettori associati alla matrice di varianza e covarianza dei dati di partenza, con gli autovalori in ordine decrescente (ovvero la prima componente principale sarà associata all'autovalore maggiore). Nel modello così descritto siamo appunto in grado di massimizzare la cosiddetta *varianza spiegata*, ovvero la somma dei primi m autovalori della matrice di varianza e covarianza. Parlando nello specifico del nostro dataset, come abbiamo visto le features presentano ordini di grandezza e unità di misura differenti: questo condiziona molto la scelta delle componenti principali, quindi bisogna standardizzare le features in modo tale che abbiano media nulla e varianza 1 e quindi confrontabili. Da notare anche che come effetto della standardizzazione i dati rientreranno automaticamente nel requisito iniziale di media nulla. Successivamente quello che facciamo è fare la PCA con $m = n$ e poi selezioniamo le m dimensioni che mantengono una percentuale di varianza spiegata accettabile.

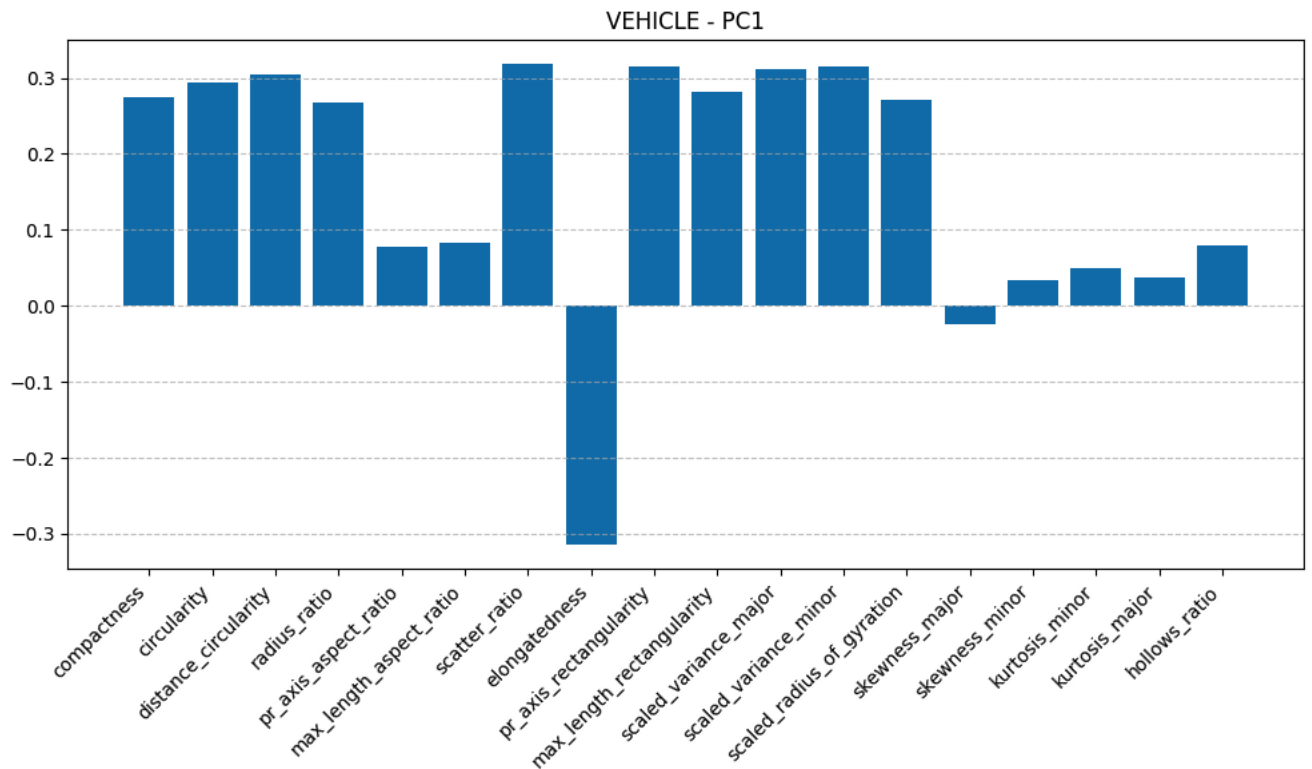
Per applicare il metodo al dataset scelgo di usare la classe *PCA* di *ScikitLearn*. Di

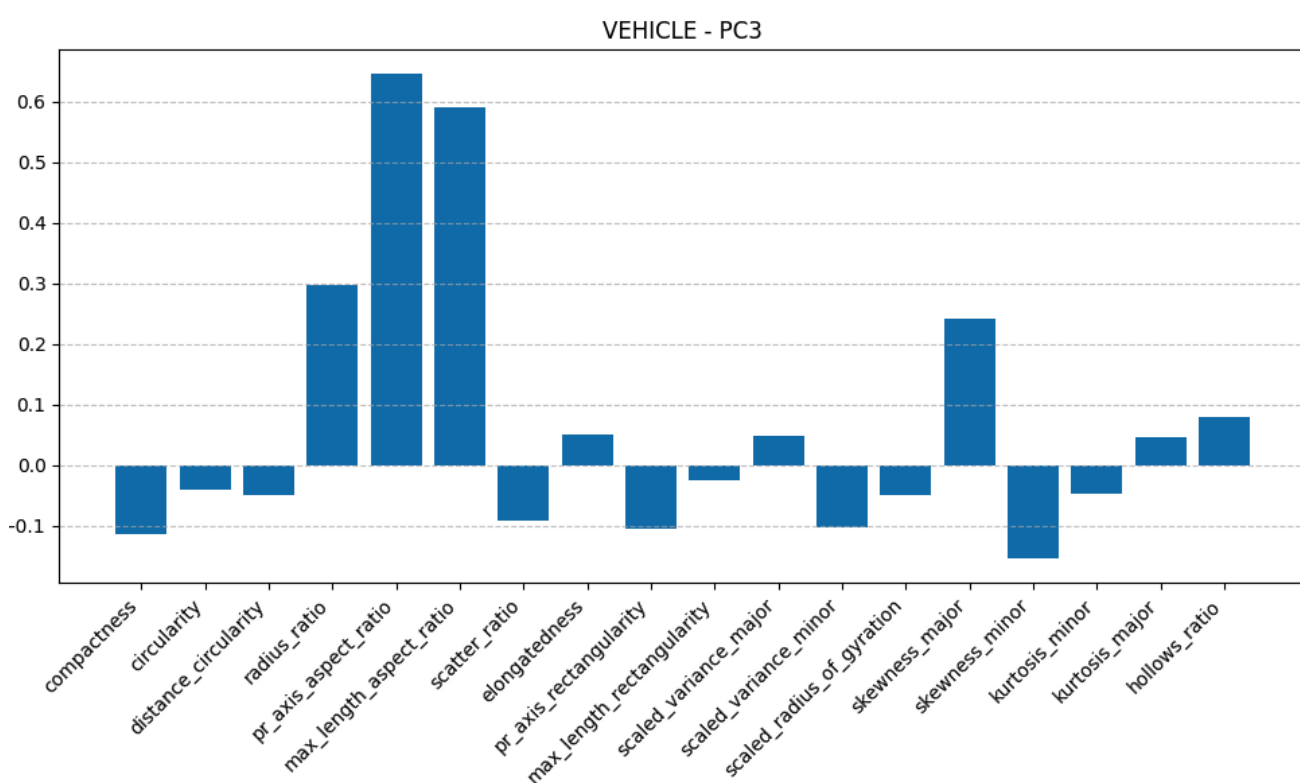
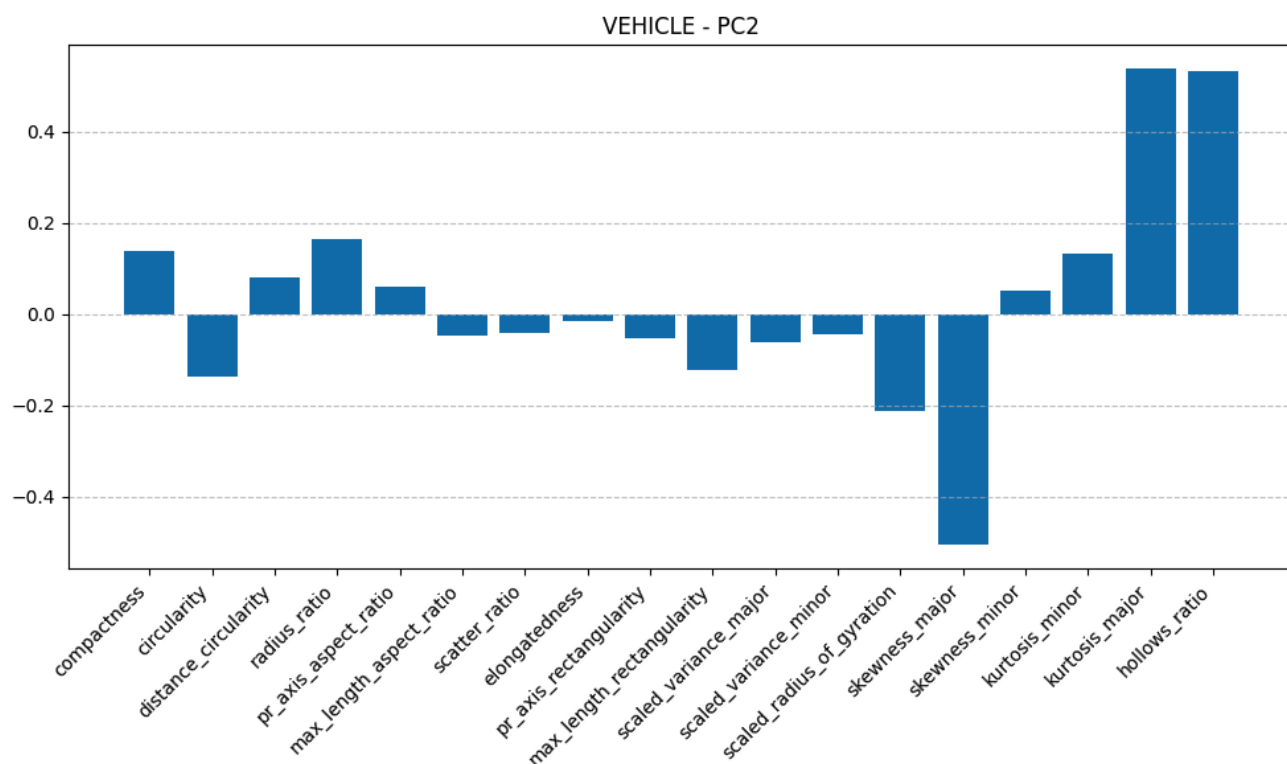
seguito i grafici che mostrano quanta varianza spiegata si mantiene all'aumentare delle componenti principali prese, pre e post riscaldamento dei dati



Dai grafici sopra si può vedere come cambia drasticamente la precisione della PCA grazie al riscalamento, in quanto probabilmente nel primo grafico PC1 è associata alla grandezza che possedeva più varianza assoluta, ovvero *scaled_variance_minor*, ottenendo così un risultato "gonfiato". Proprio per questo da ora in avanti verranno considerati solo i dati riscalati. Inoltre possiamo vedere dal secondo grafico che prendendo le prime 10 componenti manteniamo circa il 98% della varianza spiegata.

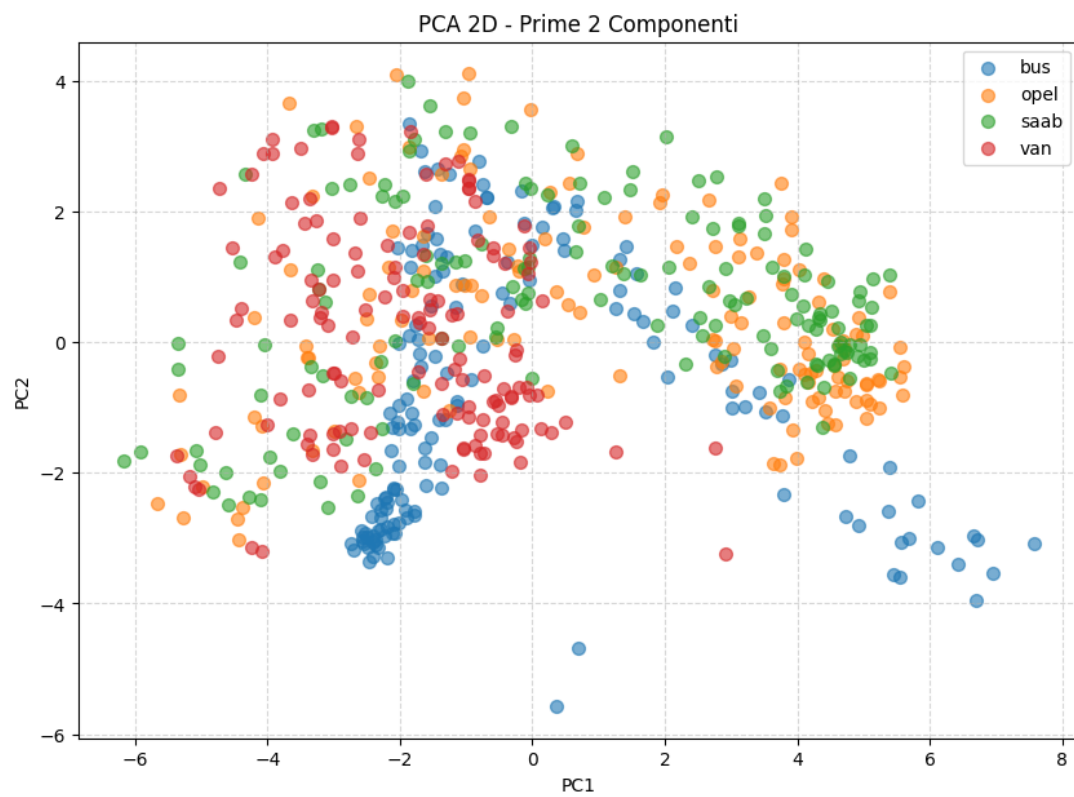
Vogliamo ora cercare di capire cosa rappresentano le prime componenti principali (ad esempio prendiamo le prime 3) valutando i contrinuti delle features ad ognuna di esse

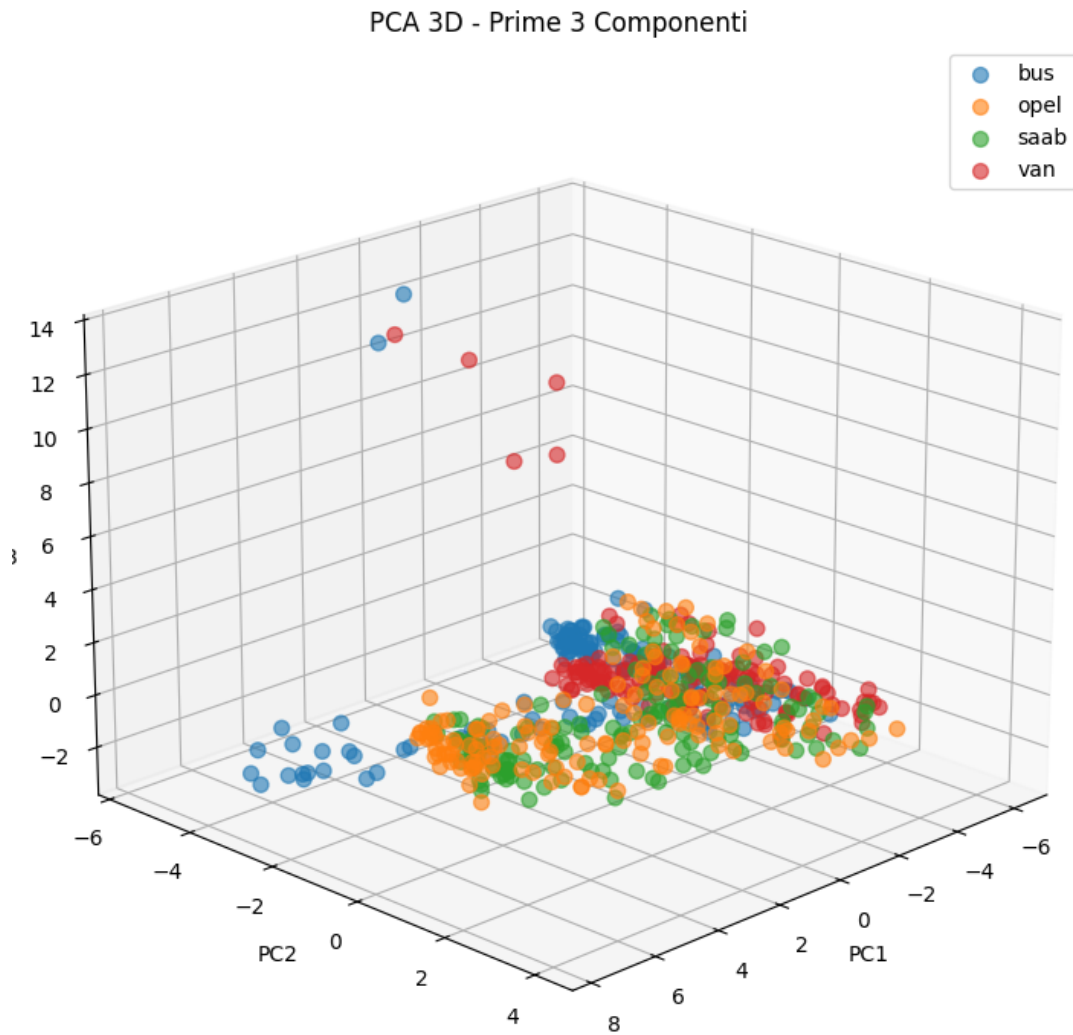




Dai grafici si può dedurre che la prima componente sia quella che riguarda maggiormente la "rotondità" generale del veicolo, mentre la terza componente si concentra sugli

assi e sui loro rapporti. Di seguito invece i grafici con i dati del training set rappresentati utilizzando prima le prime 2 componenti principali e poi le prime 3 componenti principali





E' immediato vedere che nè 2 nè 3 componenti sono sufficienti ad avere una classificazione corretta

2.2 MDA

Con la PCA avevamo la possibilità di tenere in considerazione la varianza dei dati proiettati, ma quando abbiamo bisogno di considerare la classe dei dati questo può non essere sufficiente. In queste situazioni si utilizza l'MDA, o FDA se le classi sono solamente 2, in cui si tenta di massimizzare la separazione tra classi e non la varianza. Iniziamo a considerare il caso in cui abbiamo solo due classi C_1 e C_2 : definiamo

$$\mu_i = \frac{1}{n_i} \sum_{x_j \in C_i} x_j, \quad s_i = \sum_{x_j \in C_i} (x_j - \mu_i)^2, \quad i = 1, 2$$

rispettivamente il centroide di ognuna delle due classi e lo scatter (ovvero la varianza riscalata di un fattore $(n-1)$) di ognuna delle due classi. L'idea è quella di trovare una direzione v lungo la quale proiettare i dati in modo tale che venga mantenuta la distinzione

tra classi, e questo lo facciamo tramite il discriminante di Fisher:

$$J(v) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2}$$

con $\tilde{\mu}_i = v^T \mu_i$ proiezione dei centroidi e $\tilde{s}_i = v^T s_i$ proiezione degli scatter. La massima separazione tra classi si otterrà dunque massimizzando $J(v)$. Per prima cosa introduciamo le matrici di scatter

$$S_i = \sum_{x_j \in C_i} (x_j - \mu_i)(x_j - \mu_i)^T, \quad i = 1, 2$$

da cui otteniamo

$$S_W = S_1 + S_2$$

$$S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

rispettivamente la *matrice di scatter within*, che definisce l'accumulo di varianza all'interno delle classi, e la *matrice di scatter between*, che misura la varianza relativa tra classi. A questo punto basta osservare che $\tilde{S}_i^2 = v^T S_i v$ è facile ottenere che

$$J(v) = \frac{v^T S_B v}{v^T S_W v}$$

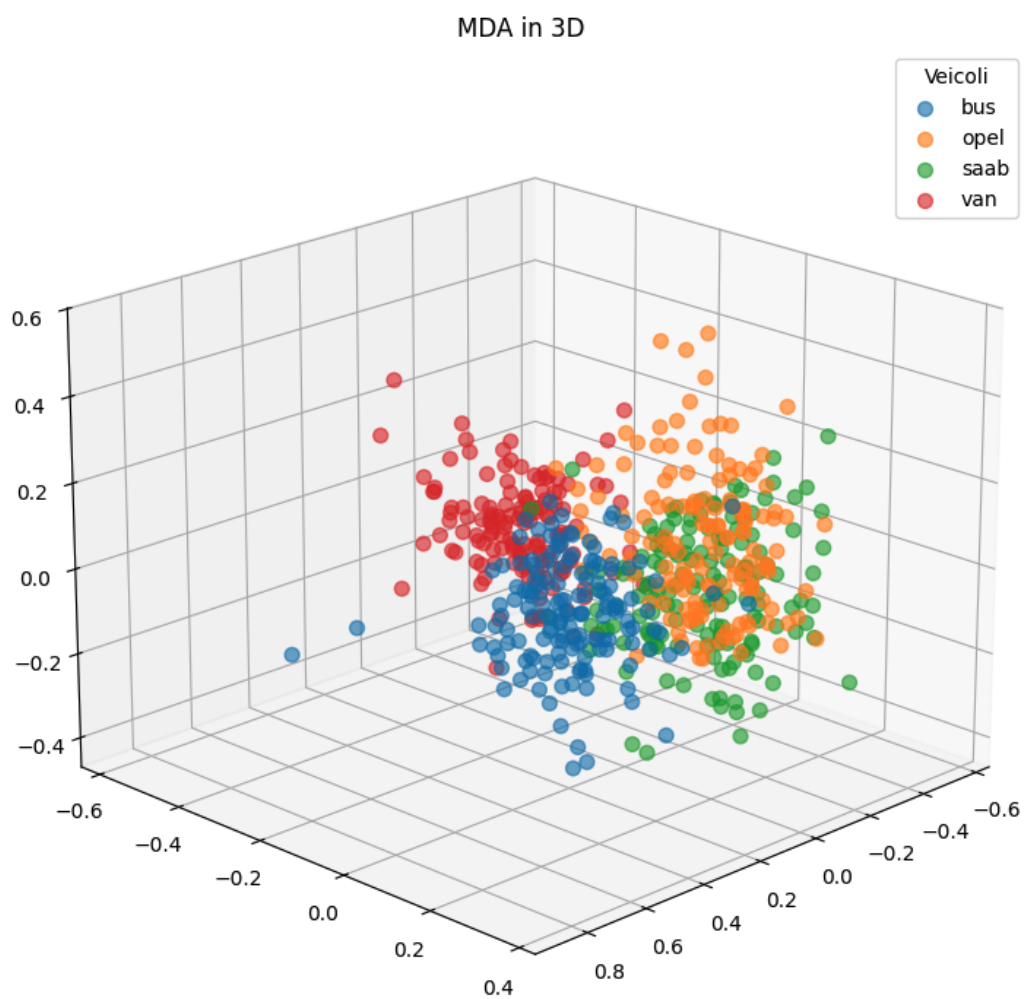
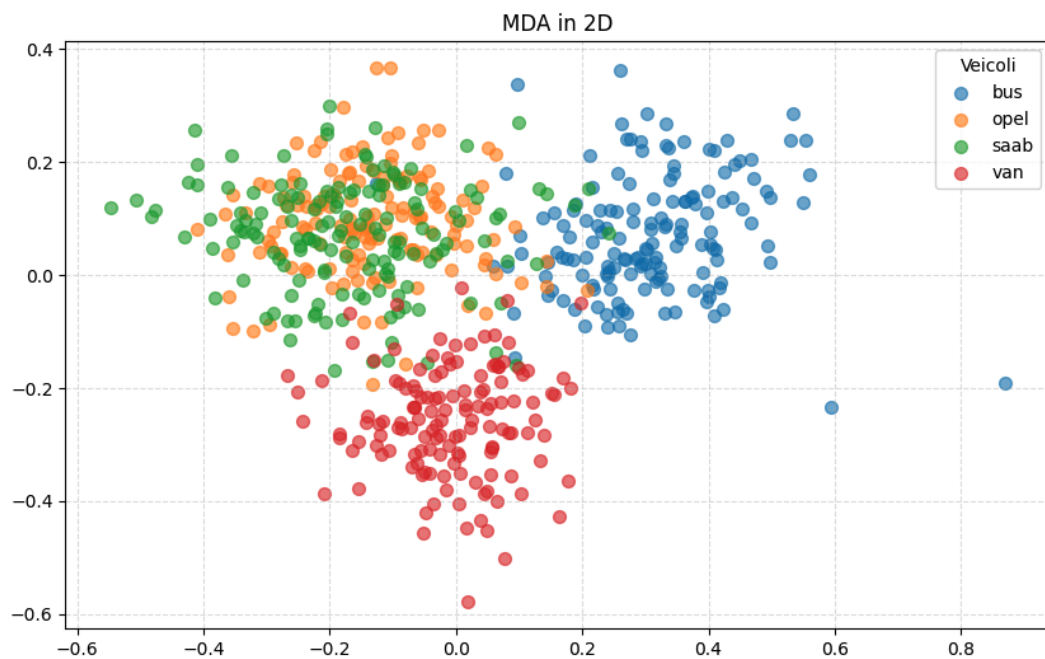
per cui massimizzare $J(v)$ diventa equivalente a cercare l'autovettore v associato al massimo autovalore del problema $S_B = \lambda S_W v$ e la rappresentazione di x_j lungo v sarà $y_j = v^T x_j$.

Quanto detto precedentemente può essere esteso a $c > 2$ classi, con l'accortezza che S_B può avere al massimo rango $c - 1$, il che per noi significa che potremmo proiettare i dati in al massimo $c - 1$ dimensioni. Il precedente problema diventa quindi un problema agli autovalori generalizzato

$$J(v) = \frac{V^T S_B V}{V^T S_W V}$$

con $V = [v_1 \dots v_k]$ matrice di proiezione contenente i primi k autovettori dei $c - 1$ trovati. La proiezione diventa quindi $y_j = V^T x_j$

Di seguito due grafici che mostrano l'MDA di *ScikitLearn* implementata sul dataset di riferimento



Quello che si può osservare soprattutto nel grafico 2D è che, come ci si aspetta, il metodo riesce sufficientemente bene a dividere *Bus* e *Van* dalle altre due classi, mentre le

ultime due sono molto mischiate tra loro: questo non stupisce se si considera che la *Opel* e la *Saab* sono dei modelli uno di macchina sportiva e l'altro di berlina, quindi a livello di silhouette molto più difficili da classificare rispetto alle altre due in quanto più simili

Capitolo 3

Classificazione

Siamo ora interessati ad implementare dei modelli che, noti i valori delle feature associati ad una variabile nuova, ne predicano correttamente la classe di appartenenza. I metodi che scegliamo di implementare sono associati a dei parametri che necessitano di addestramento, per questo si è scelto di suddividere il dataset come descritto all'inizio.

Per valutare l'efficacia dei vari classificatori ho utilizzato il modulo *metrics* di *ScikitLearn*. Nello specifico ho utilizzato i seguenti metodi di valutazione

- **Matrice di Confusione:** matrice quadrata (nel nostro caso 4×4) che confronta le etichette reali con quelle predette dal modello. Permette di individuare visivamente quali classi vengono confuse più frequentemente

Siano ora TP = numero di true positives predetti, FP = numero di false positives predetti, TN = numero di true negatives predetti e FN = numero di false negatives predetti

- **Accuracy Score:**

$$\frac{TP + TN}{TP + TN + FP + FN}$$

rappresenta la frazione totale di predizioni corrette sul totale dei campioni analizzati. È l'indice globale di esattezza del modello.

- **Precision Score:**

$$\frac{TP}{TP + FP}$$

Misura la capacità del modello di non classificare come positivo un campione che in realtà è negativo

- **Recall Score:**

$$\frac{TP}{TP + FN}$$

Misura la capacità del modello di individuare tutti i campioni positivi presenti nel dataset

- **F1-score**

$$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

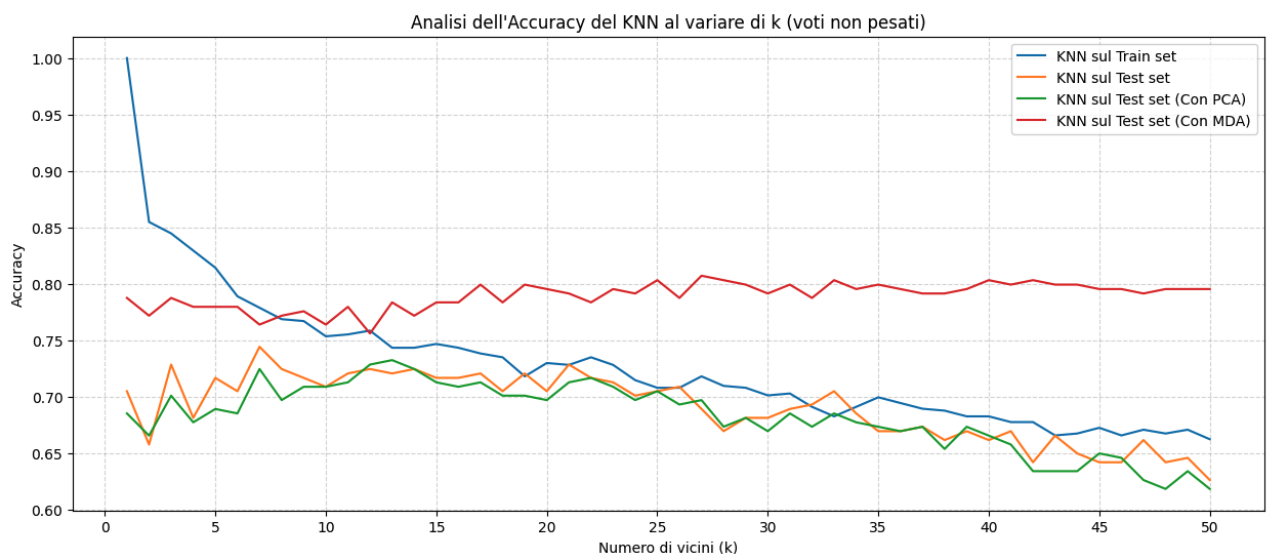
Media armonica tra Precision e Recall

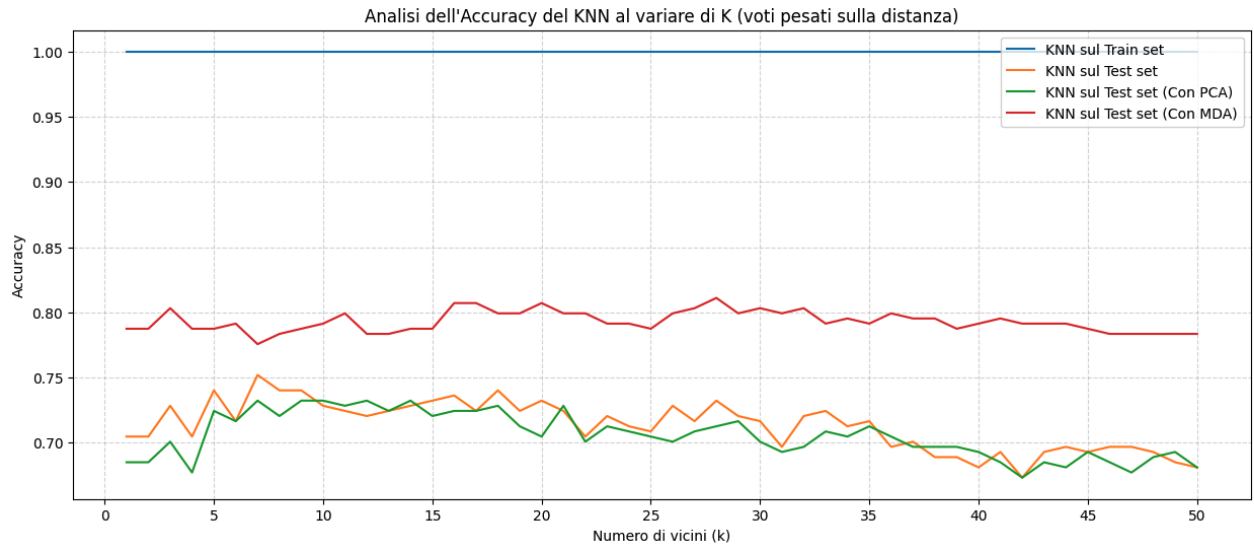
Per i calcoli di Precision, Recall e F1-score, siccome ci troviamo davanti ad un problema di classificazione multiclasse, ho applicato il parametro `average='weighted'`. Questa configurazione calcola lo score per ciascuna classe indipendentemente, per poi effettuarne una media pesata in base al numero di campioni reali presenti in ogni classe

3.1 KNN

Il K^{th} *Nearest Neighbours* (KNN) è un metodo che predice la classe di appartenenza di un dato in base ai k dati più vicini. In base a questi dati, ad ogni classe presente viene dato un voto: la classe con il voto più alto sarà quella scelta per il dato di partenza. Un approccio efficace per evitare problemi di pareggi è quello di considerare i voti pesati sulle distanze dal dato, quindi più il k -esimo punto sarà distante dal nostro dato, meno contribuirà al voto finale di quella classe.

Per l'implementazione sul dataset ho utilizzato il *KNeighborsClassifier* di *ScikitLearn* allenato sul training set e poi testato sul test set (questo per i dati sia raw, che proiettati con PCA, che proiettati con MDA). Per quanto riguarda i parametri ho scelto di mantenere come loss la distanza euclidea standard e ho scelto di rappresentare cosa succede all'accuracy sia senza pesare i voti assegnati ad ogni classe, sia pesandoli sulla distanza dai k dati. Di seguito i grafici:

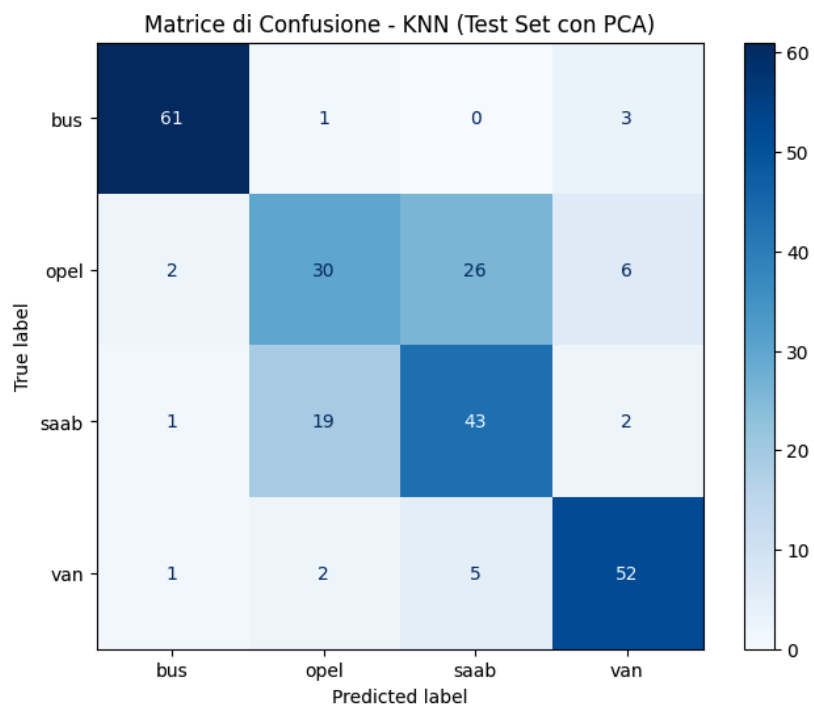
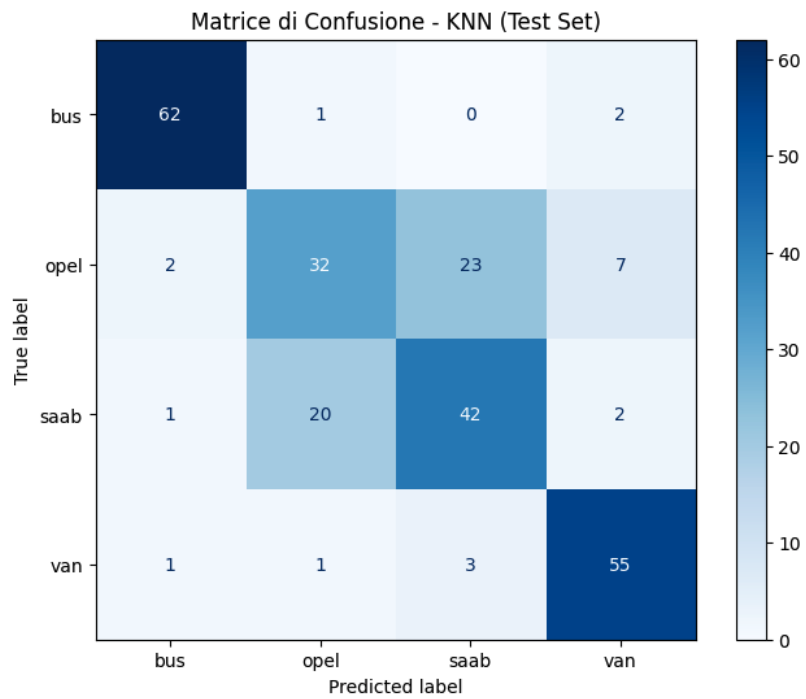


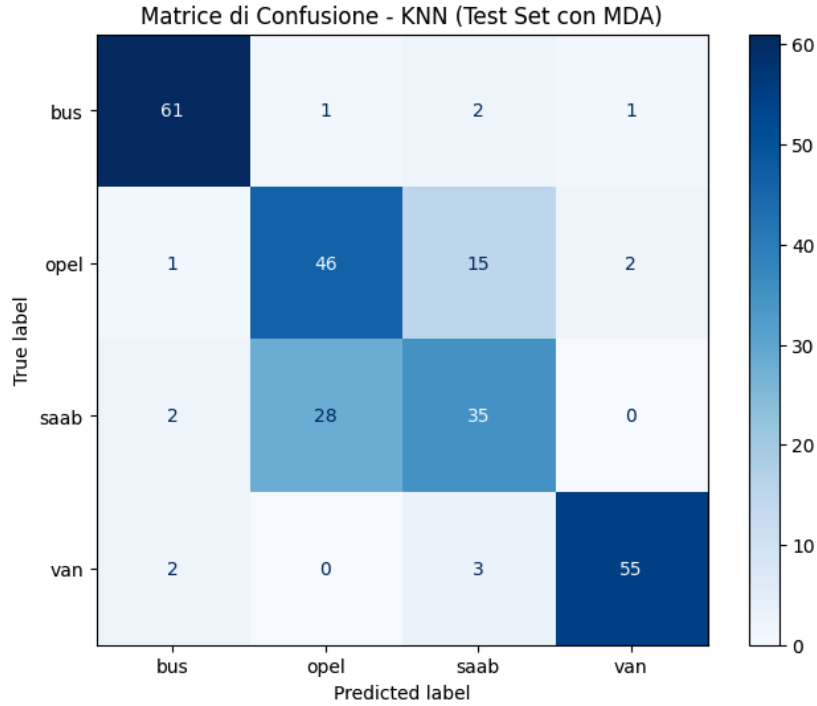


Si osserva immediatamente la differenza di accuratezza sul training set quando si pesano i voti o meno e si nota inoltre che per $k < 5$ siamo chiaramente in zona di overfitting, poichè osservando il primo grafico vediamo che l'accuratezza sul training è elevata mentre sui dati di test è bassa e tende ad aumentare, per $5 < k < 10$ ci troviamo in una zona ottimale e oltre tutte le accuratèzze tendono a crollare. Scelgo allora $k = 7$ per mostrare gli score generali da comparare con gli altri classificatori

	KNN sul Train	KNN sul Test	KNN sul Test (PCA)	KNN sul Test (MDA)
Accuracy	1.0	0.751969	0.732283	0.775591
Precision	1.0	0.744620	0.729201	0.777911
Recall	1.0	0.751969	0.732283	0.775591
F1-score	1.0	0.746741	0.728547	0.774578

Questi dati comunicano innanzitutto che la riduzione di dimensionalità sul dataset per questo classificatore è stata molto efficace, tuttavia possono stupire se si considera che per il nostro caso specifico l'MDA proietta in 3 dimensioni siccome le classi sono 4, ovvero la perdita di informazione è elevata e ci si aspetterebbe un risultato peggiore degli altri sul test set. Per fare chiarezza osserviamo anche le matrici di confusione





Le matrici di confusione ci comunicano chiaramente qual è il problema: come si era supposto osservando i grafici dell'MDA il problema più grande è classificare le *Saab* e le *Opel*, siccome non sono facilmente linearmente separabili in quanto hanno caratteristiche simili. Pertanto l'MDA che tiene conto della separazione tra classi per proiettare ottiene risultati migliori della PCA

3.2 LDA

La *Linear Discriminant Analysis (LDA)* è un metodo di supervised learning che sfrutta il teorema di Bayes per predire l'appartenenza ad una specifica classe di una variabile x . Più nello specifico se consideriamo x come la realizzazione di una variabile aleatoria X e considero k la realizzazione della variabile aleatoria Y , che rappresenta la classe scelta dall'algoritmo, sono interessato a trovare con che probabilità x finirà nella classe k , ovvero

$$\mathbb{P}(Y = k | X = x) = \frac{\mathbb{P}(X = x | Y = k) \cdot \mathbb{P}(Y = k)}{\mathbb{P}(X = x)} = \frac{\pi_k(x)f_k(x)}{\sum_{i=1}^c \pi_i(x)f_i(x)},$$

dove $f_k(x)$ è la densità di probabilità della classe k , mentre $\pi_k \in (0, 1)$ è la probabilità a priori di appartenere alla classe k . In questo contesto x verrà assegnata alla classe k che massimizza $\mathbb{P}(Y = k | X = x)$: per fare ciò è necessario assumere che le densità di distribuzione nelle classi siano determinate a priori, nel nostro caso assumiamo siano normali, e che siano omoschedastiche, ovvero varianza uguale per ognuna

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|S|^{1/2}} e^{-\frac{1}{2}(x-\mu_k)^T S^{-1}(x-\mu_k)}$$

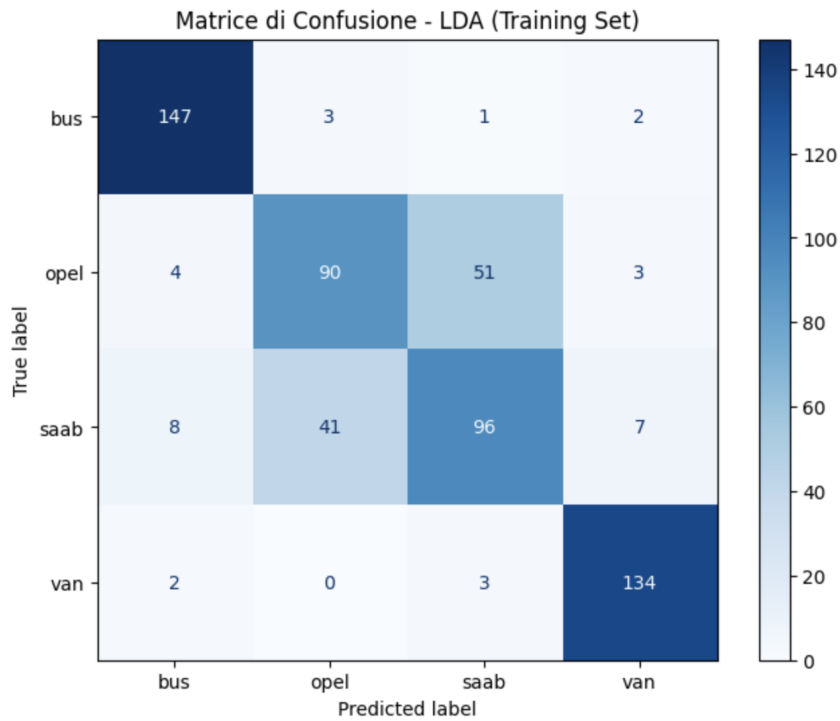
L'obiettivo è quindi massimizzare questa probabilità e per farlo si procede massimizzando il logaritmo, ottenendo la funzione discriminante

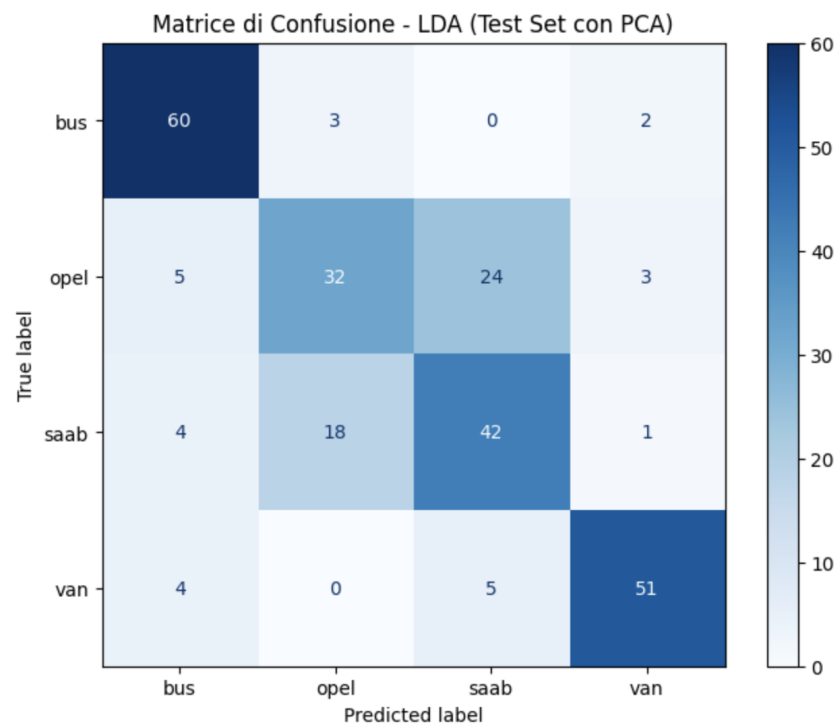
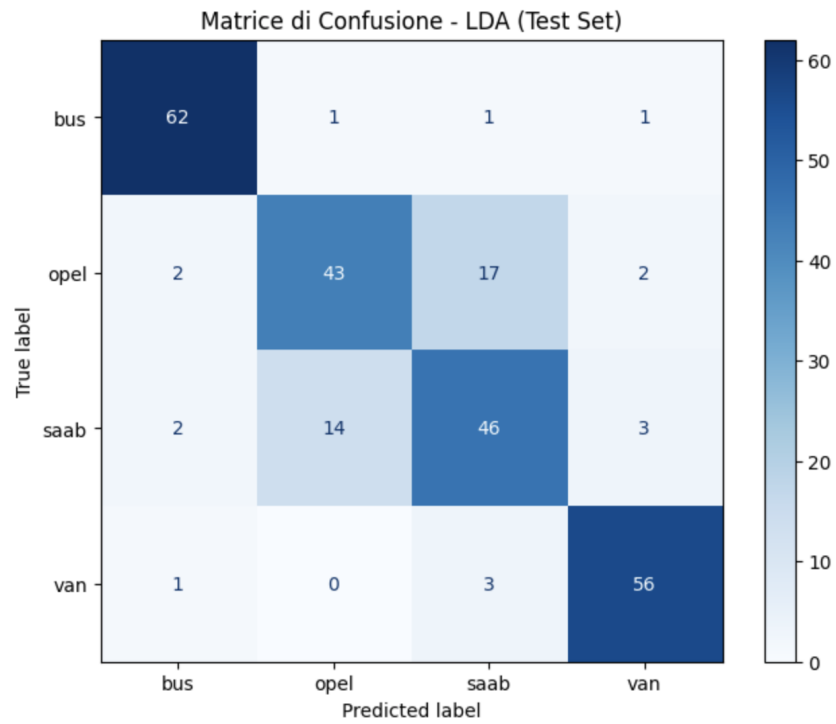
$$\delta_k(x) = xS^{-1}\mu_k - \frac{1}{2}\mu_k^T S^{-1}\mu_k + \log \pi_k, \quad \forall k = 1, \dots, c,$$

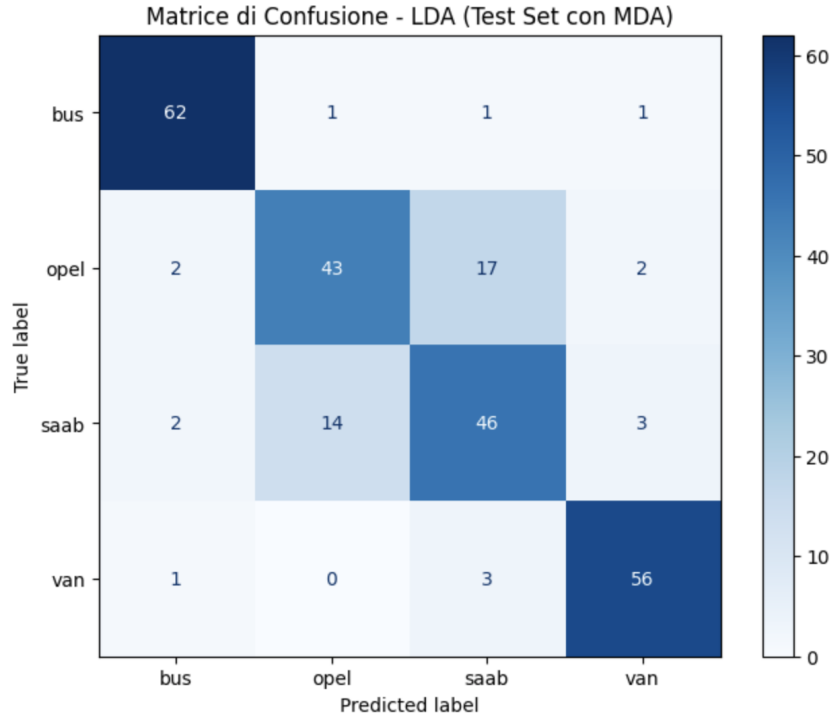
dove S_k dovrebbe essere la matrice di varianza-covarianza dei punti appartenenti alla classe k , tuttavia per l'omoschedasticità $S_1 = \dots = S_c = S$.

Il metodo viene implementato tramite *ScikitLearn* e viene prima allenato sul training set e poi testato sul test set. Di seguito gli score ottenuti dal metodo e la matrice di confusione del test set

	LDA sul Train	LDA sul Test	LDA sul Test (PCA)	LDA sul Test (MDA)
Accuracy	0.788851	0.814961	0.728346	0.814961
Precision	0.782618	0.812669	0.725201	0.812669
Recall	0.788851	0.814961	0.728346	0.814961
F1-score	0.785048	0.813230	0.724351	0.813230







L'osservazione più interessante è la bassa precisione sui dati di training: questo non stupisce se si considera che l'LDA utilizza solo funzioni lineari per separare le classi, cosa che osservando l'MDA sappiamo non essere molto efficace. Tuttavia questo molto probabilmente garantisce che non ci sia overfitting sui dati di test, ed effettivamente gli score sui dati di test sono piuttosto elevati

3.3 SVM

Le *Support vector machines (SVM)* sono uno strumento di classificazione lineare e binaria che ha come obiettivo quello di scegliere un iperpiano $h : X \rightarrow \{-1, 1\} \in H$, dove $H = \{x \rightarrow \text{sgn}(\langle w, x \rangle + b) \text{ t.c. } w \in \mathbb{R}^n, b \in \mathbb{R}\}$ per cui l'errore di generalizzazione $R(h)$ sia minimo. Nello specifico questo lo facciamo massimizzando il margine, ovvero la distanza tra l'iperpiano e il dato più vicino

$$r = \min_{i=1, \dots, N} d(h, x_i) = \min_{i=1, \dots, N} \frac{|\langle w, x_i \rangle + b|}{\|w\|}$$

Quindi, indicando con $y_i \in \{-1, 1\}$ la classe del dato x_i , il nostro obiettivo sarà

$$\max_{w, b : y_i(\langle w, x_i \rangle + b) \geq 0} \min_{i=1, \dots, N} \frac{|\langle w, x_i \rangle + b|}{\|w\|}$$

che conduce al problema di ottimizzazione

$$\begin{cases} \min_{w, b} \frac{1}{2} \|w\|^2 \\ \text{s.t. } y_i(\langle w, x_i \rangle + b) \geq 1, \quad i = 1, \dots, N \end{cases}$$

I *support vector* saranno proprio i dati che si trovano sugli iperpiani a distanza $\frac{1}{\|w\|}$ da h cercato. In generale questo funziona, ma se siamo interessati a conferire una maggiore

adattabilità al sistema dobbiamo introdurre delle cosiddette variabili di *slack* ξ_i che sono associate ad un costo quando un dato finisce dentro un margine

$$\begin{cases} \min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t. } y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad i = 1, \dots, N, \xi_i \geq 0 \end{cases}$$

3.3.1 Kernel SVM

Quanto detto ovviamente funziona sotto l'assunzione che le due classi siano linearmente separabili in partenza. Se vogliamo generalizzare l'approccio a dataset non linearmente separabili, sarà necessario mappare i nostri dati in uno spazio di dimensione superiore dove sono linearmente separabili. Questo è possibile grazie alla teoria sugli spazi di Hilbert e le funzioni chiamate *kernel*, $K : X \times X \rightarrow \mathbb{R}$ t.c $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$. I kernel K in particolare sono delle matrici simmetriche e definite positive. Queste funzioni sono molto utili perchè se si passa al problema duale di ottimizzazione dell'SVM, si può notare che dipende solamente da prodotti scalari: quindi se noi troviamo una mappa $\phi(x)$ che mandi i dati in uno spazio di dimensione superiore in cui è possibile separarli con una funzione affine, allora possiamo utilizzare i valori $\phi(x_1), \dots, \phi(x_N)$ per fare l'SVM con la sicurezza di avere dei dati linearmente separabili, ma senza computare veramente uno spazio di dimensione superiore che può essere potenzialmente molto alta. Quello che si ottiene è poi una funzione non lineare dello spazio di partenza che separa i dati

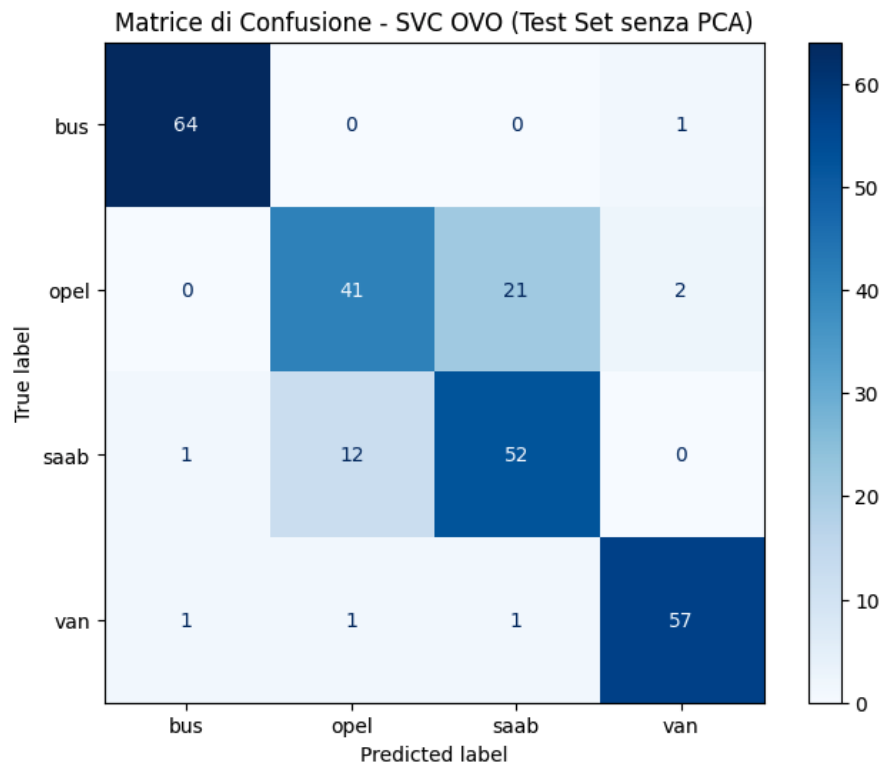
3.3.2 SVM multiclasse

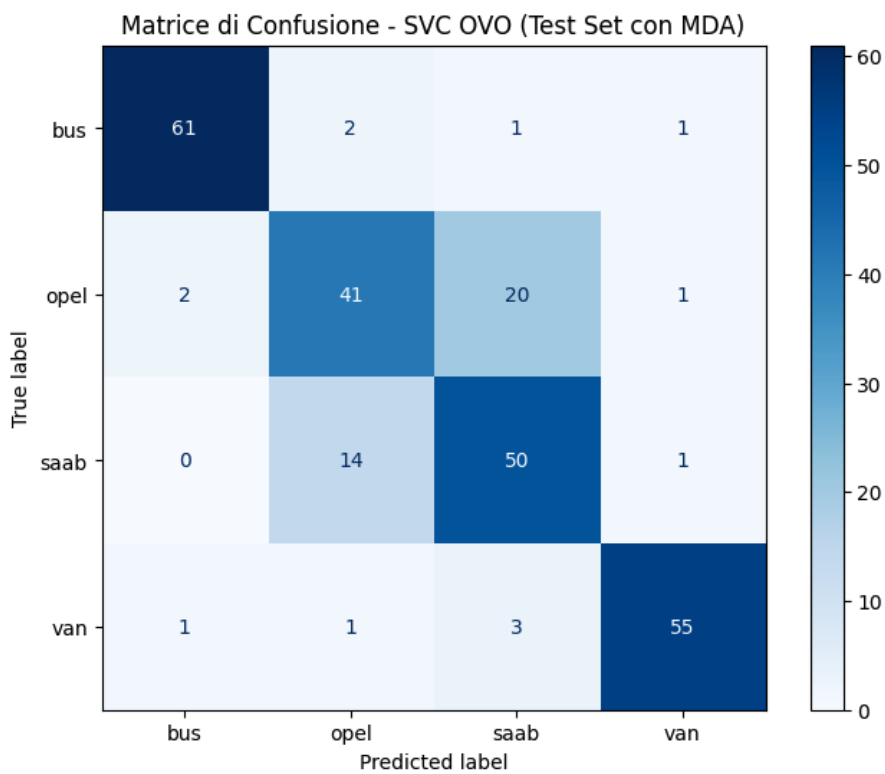
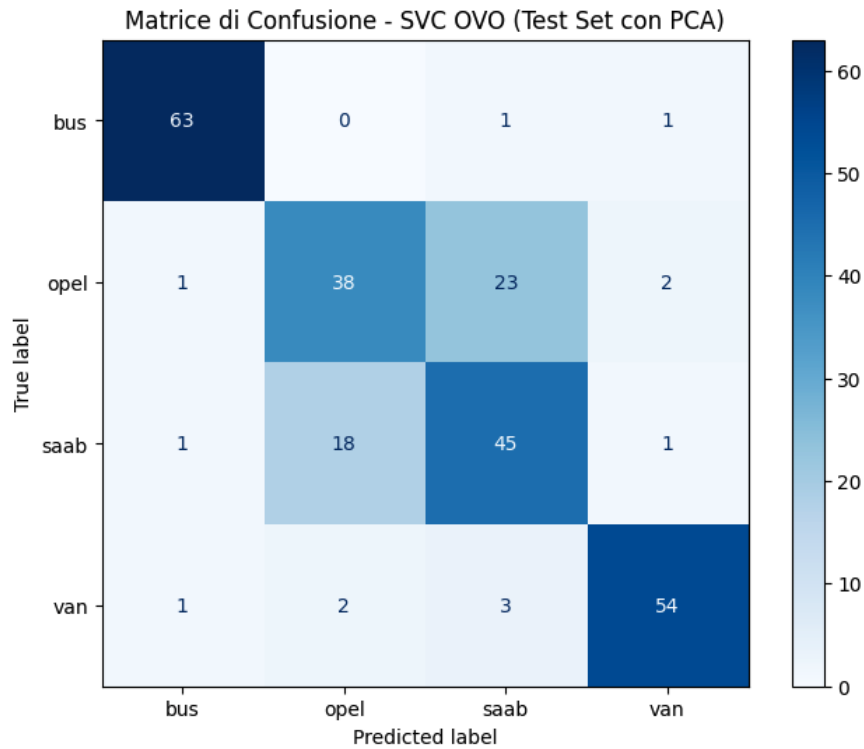
Nel caso specifico in cui la classificazione non sia binaria ma multiclasse come nel nostro dataset, bisogna generalizzare l'SVM. Per fare ciò abbiamo due approcci possibili:

1. **One-Vs-One (OVO)**: come suggerisce il nome consiste nell'applicare l'SVM sul training set a coppie di classi fino a esaurire le combinazioni possibili. Poi in fase di training ogni classificatore ottenuto dà la sua predizione sul test set per ogni dato, la classe predetta in numerosità maggiore sarà quella considerata come predizione del metodo da confrontare con quella reale
2. **One-Vs-All (OVA)**: nella fase di training viene applicata l'SVM binaria considerando una classe come positiva e considerando tutte le altre assieme come una classe negativa. Successivamente, nella fase di test, ogni classificatore conferisce un grado di probabilità con cui il dato in esame appartiene ad ognuna delle classi: la classe che raggiunge lo score maggiore è quella predetta da confrontare con quella reale

Nell'applicazione pratica al dataset in esame, si è utilizzato l'*SVC* di *ScikitLearn* per fare l'SVM non lineare utilizzando un kernel di tipo RBF, ovvero di tipo esponenziale, mentre gli iperparametri sono stati scelti per tentativi arrivando ad un compromesso soddisfacente. Dato il numero ridotto di dati presenti, la scelta è stata di utilizzare il paradigma OvO per trovare i vari SVM della tipologia sopra descritta. Di seguito sono riportati gli score ottenuti dal metodo sui dati di training e sui dati di test originali, proiettati con PCA e proiettati con MDA e le relative matrici di confusione

	One-vs-One Train	One-vs-One	One-vs-One (PCA)	One-vs-One (MDA)
Accuracy	0.913851	0.842520	0.787402	0.814961
Precision	0.913332	0.843695	0.789227	0.818937
Recall	0.913851	0.842520	0.787402	0.814961
F1-score	0.913523	0.841020	0.787417	0.815685





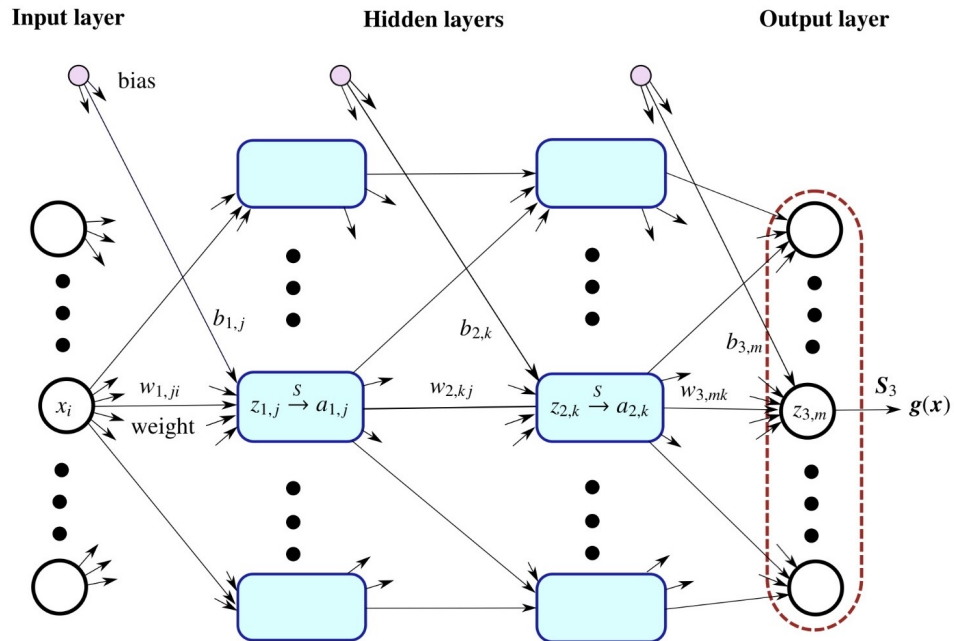
Come si può notare, il metodo si comporta molto bene in tutti e quattro i casi e in accordo con quanto emerge osservando l'MDA in due dimensioni, le classi più difficili da distinguere e dunque da classificare correttamente risultano nuovamente *Opel* e *Saab*

3.4 MLP

Il *Multi Layer Perceptron* è un particolare tipo di rete neurale della tipologia *feed forward*. In queste reti neurali, l'apprendimento avviene grazie ad una serie di composizioni di una funzione specifica $\{S_1, \dots, S_L\}$ con L numero di layer, chiamata *funzione di attivazione*, applicata ai dati che però ad ogni step vengono trasformati da una funzione affine con due parametri: $W_l = [w_{l,ij}]$ matrice dei pesi e $b_l = [b_{l,j}]$ vettore dei bias. Possiamo quindi ottenere la nostra funzione di output desiderata $g(x)$ nel modo seguente:

$$x \rightarrow S_1(W_1x + b_1) = a_1 \rightarrow S_2(W_2a_1 + b_2) = a_2 \rightarrow \dots \rightarrow S_L(W_La_{L-1} + b_L) = g(x)$$

Questo processo è chiamato appunto *feed forward propagation* e può essere rappresentato da un grafo orientato aciclico fatto a strati dove il numero di layers L è chiamata profondità della rete, mentre se chiamiamo p_l il numero di nodi dell' l -esimo layer, $\max_l p_l$ è detta ampiezza della rete. Il primo strato corrisponde a quello degli input x , mentre l'ultimo a quello dell'output $g(x)$ e quelli in mezzo sono detti *hidden layers*. In generale gli hidden layers possono avere un numero di nodi variabile, ma tendenzialmente si considerano aventi lo stesso numero di nodi. Quanto detto può essere rappresentato graficamente dal seguente schema



Per quanto riguarda l'applicazione al dataset, si è utilizzato il pacchetto di *ScikitLearn* chiamato *MLPClassifier*. L'apprendimento avviene associando alla funzioni in output della rete una *loss* rispetto ad una specifica funzione obiettivo ricercata e aggiornando i pesi per cercare di minimizzarla. Più nello specifico, viene suddiviso il training set in sottoinsiemi più piccoli chiamati minibatch, e su ognuno di questi si valuta l'errore medio fatto dalla rete sulla singola minibatch e si aggiustano i pesi. Il metodo che ho utilizzato usa un algoritmo estrapolato dallo *stochastic gradient descent* per farlo, ovvero

calcola il gradiente della loss e si muove nella direzione opposta per aggiornare i pesi. In questo contesto possiamo definire il *learning rate* della rete proprio come il passo del metodo del gradiente applicato. Una volta che la rete ha analizzato tutte le minibatch che compongono il training set e ha aggiornato i pesi di conseguenza, avrà compiuto un'epoca. Siccome la rete aggiorna i pesi a piccolissimi passi, saranno necessarie molte epoche per avere un risultato ottimale

Nello specifico la scelta dei parametri è stata fatta manualmente facendo dei test su dei valori scelti come segue:

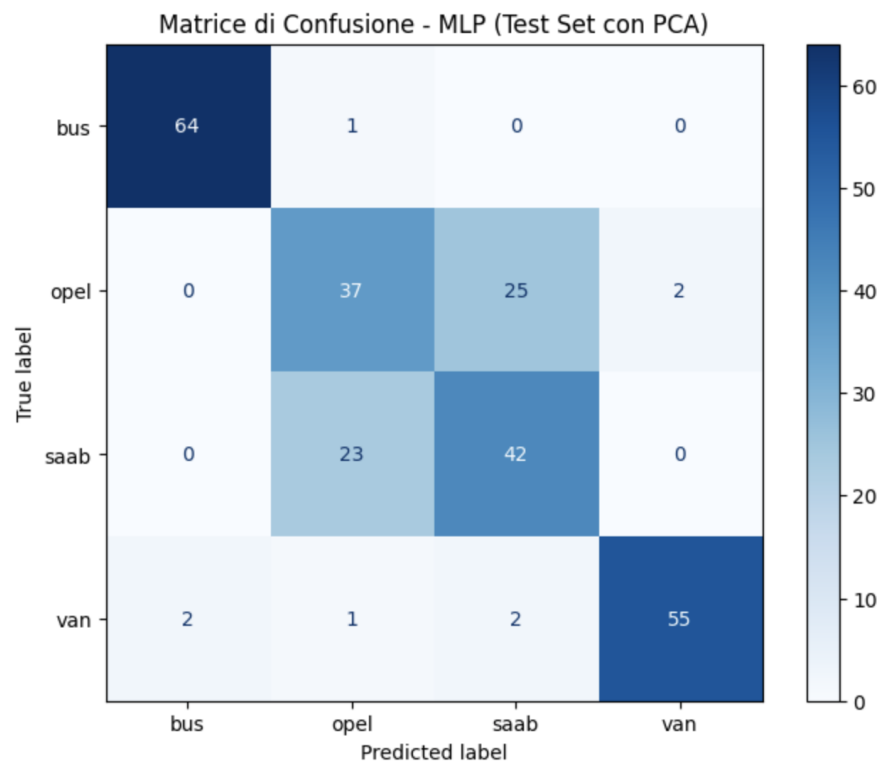
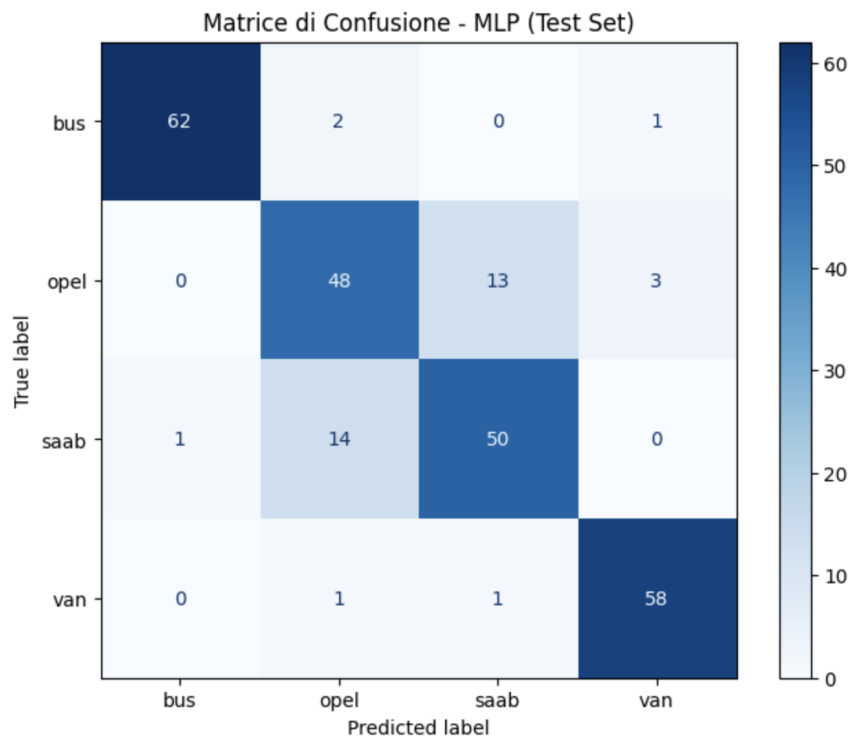
- Ho scelto come funzione di attivazione la Rectified Linear Unit ('relu'), ovvero

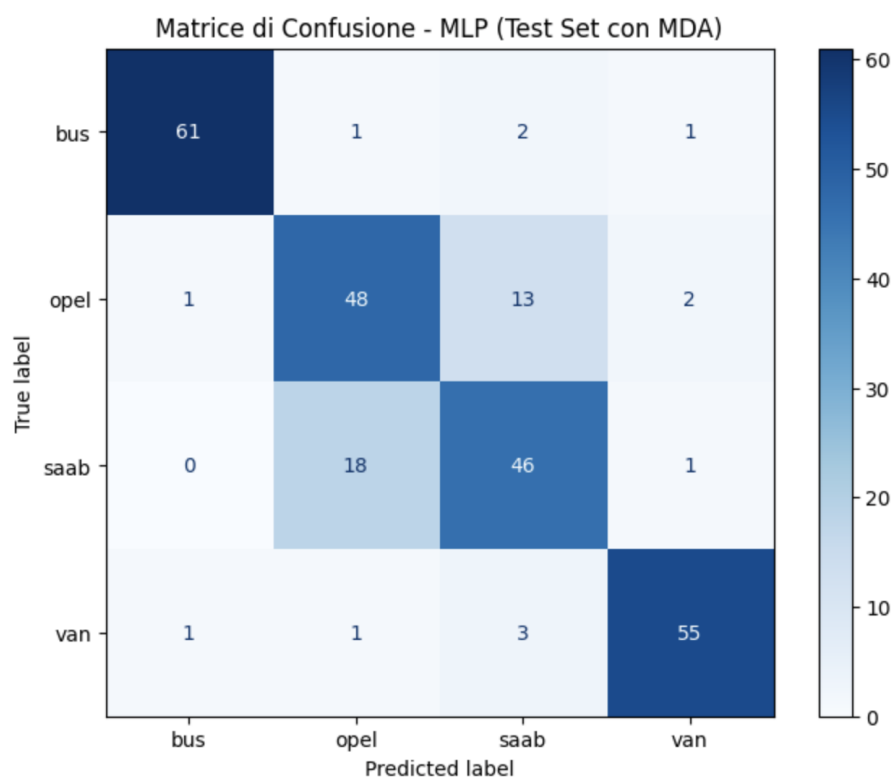
$$f(x) = \max(0, x)$$

- Ho impostato il solver sul metodo 'adam', ovvero un metodo molto simile allo stochastic gradient descent ma con un learning rate non fisso dipendente dai gradienti e dagli aggiornamenti del passo precedente
- Provando a testare con varie numerosità, è stato evidente che gli score migliori erano ottenuti usando un solo hidden layer. Per quanto riguarda il numero di nodi ho fatto vari test tenendomi intorno a multipli delle features e ho trovato in 100 un valore ottimale
- Ho riscontrato che 700 epoche fossero sufficienti per ottenere una loss sufficientemente piccola su tutti i metodi, con un limite di 20 epoche successive in cui la loss non cambia più della tolleranza di default per stoppare il processo prima del raggiungimento delle 700 epoche totali
- Tenendo in considerazione la cardinalità di ogni insieme considerato ho individuato che le minibatch di grandezza 54 fossero ottimali

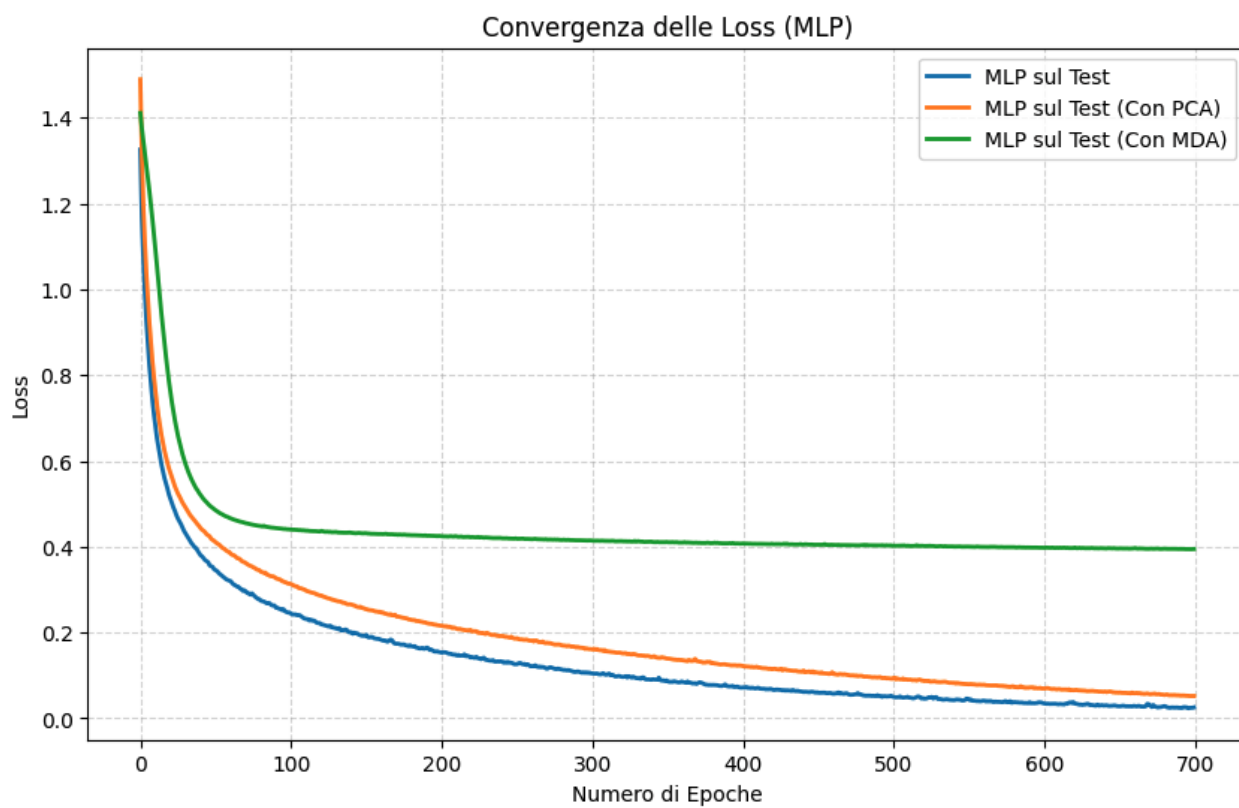
Di seguito la tabella degli score ottenuti dalla rete così costruita e le matrici di confusione

	MLP sul Train	MLP sul Test	MLP sul Test (Con PCA)	MLP sul Test (Con MDA)
Accuracy	1.0	0.858268	0.779528	0.826772
Precision	1.0	0.858819	0.782220	0.829779
Recall	1.0	0.858268	0.779528	0.826772
F1-score	1.0	0.858399	0.780533	0.828021





Infine è interessante osservare cosa succede alle loss con lo scorrere delle epoche



E' possibile osservare infatti che tutte e 4 convergono piuttosto rapidamente, ma la loss sui dati di test proiettati con MDA non scende sotto 0.4. Questo può stupire se si considera che gli score ottenuti dal metodo su quel set sono in realtà piuttosto alti, ma risulta evidente se si considera anche la matrice di confusione: infatti nuovamente vediamo che anche questo metodo fatica maggiormente a classificare le classi *Opel* e *Saab* e questo causa che in fase di training l'MLP deve trovare dei bound più complessi per adattarsi ai dati proiettati con PCA in quanto più mescolati, mentre sui dati proiettati con MDA c'è maggiore linearità a priori quindi i bound saranno più semplici ma farà più fatica a prevedere le due classi mescolate, e questo causa una loss che non tende propriamente a 0 seppur convergendo ma un'accuracy elevata essendo le due classi molto mischiate e quindi probabilisticamente più facile classificare i dati correttamente

Capitolo 4

Conclusioni

In generale l'MLP si è rivelato il classificatore migliore per il nostro dataset (con questa configurazione specifica dei dati) su 3 dei 4 insiemi considerati, tranne sui dati di test proiettati con PCA, su cui invece l'SVC sembra essersi comportato leggermente meglio ma di un ordine di grandezza che può essere dovuto alla scelta dei parametri più che all'efficacia del metodo in sè.

Si può notare in generale che i classificatori non hanno perso troppi punti percentuali lavorando con i dati proiettati con MDA rispetto a quelli non proiettati, mentre invece la PCA ci dà degli score più negativi. Inoltre in accordo con quanto abbiamo osservato con l'MDA, tutti i classificatori hanno mostrato più difficoltà nel distinguere la classe *Opel* dalla *Saab* a causa delle loro similitudini, il che spiega anche perchè gli score sui dati proiettati con PCA sono generalmente peggiori

4.1 Bibliografia

1. Chaudhary, L. (2018). Classifying-silhouettes-of-vehicles: Unsupervised Learning (PCA) on Vehicle dataset (Jupyter Notebook) [Repository GitHub].
2. Hastie, T., Tibshirani, R., e Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer Science e Business Media.
3. Shalev-Shwartz, S., e Ben-David, S. (2014). Understanding machine learning: From theory to algorithms. Cambridge University Press.